

NeoGenCode Technologies Flutter Developer - What to Learn

MUST Learn (Class A keywords with no direct project experience)

- **Flutter & Dart** – your mobile experience is React Native (JavaScript/TypeScript), not Flutter (Dart). Both are cross-platform frameworks with a component/widget tree and hot reload, so the mental model transfers – React components $\approx$ Flutter widgets, JSX $\approx$ Dart's declarative widget syntax. Dart itself is a much smaller lift than a new paradigm: it's a statically-typed, C-style OOP language. Build one small Flutter app (a to-do list or similar) to get comfortable with the widget tree, StatefulWidget vs StatelessWidget, and Dart syntax before interviewing.
- **BLoC state management** – Flutter's most common state-management pattern (Business Logic Component), built around Streams and event/state pairs. Bridge from Redux (which you know from Clink AI): both centralize state and separate business logic from UI, but BLoC uses reactive Streams instead of Redux's reducer/action pattern. Once you have a basic Flutter app running, add BLoC to one screen to see the pattern directly.
- **Clean Architecture (in a Flutter context)** – a layered architecture pattern (presentation / domain / data layers) for structuring larger apps for testability and separation of concerns. You've practiced architectural thinking in Echo's multi-tenant monorepo design; Clean Architecture formalizes that same instinct into named layers with strict dependency rules (inner layers never depend on outer ones).

GOOD to Learn (strengthens the application, not blocking)

- **Firebase** – Google's mobile backend-as-a-service (auth, Firestore, push notifications, analytics). You have real backend/auth experience (Clerk, JWT/OAuth across your projects); Firebase is a specific vendor's version of similar building blocks, commonly paired with Flutter apps.
- **Flutter animations & advanced widgets** – once comfortable with the basics, explore Flutter's animation APIs (AnimationController, implicit animations) and custom widget composition – polish that differentiates a Flutter developer but isn't foundational.
- **Automated testing for Flutter** – widget tests and integration tests using Flutter's built-in test framework. None of your current projects have test suites, so this would be new practice regardless of language, but worth a light pass given it's explicitly named as a plus.

Fast-Ramp Note

This JD wants Flutter/Dart specifically, and your production mobile experience is entirely React Native – a real language and framework switch, though the underlying cross-platform mobile concepts (widget/component trees, state management, platform-specific debugging on iOS/Android) transfer directly from your Clink AI and Edfora experience. Prioritize a small Flutter + BLoC project before interviewing, since "2+ years professional Flutter/Dart" is a must-have qualification you don't currently meet on paper – be ready to speak to your React Native depth as the closest analogue.